Lab Experiment 4- Designing a Pipelined RISC Processor (Part 1)

Objective

The objective of this lab is for you to build the building blocks required to construct a pipelined RISC (Reduced Instruction Set Computer) processor capable of executing 16 different instructions including load, store, arithmetic and logical operations. The processor will not perform branch and control instructions. You will design and implement the functional units of the processor, including the instruction unit, instruction decode, instruction execution, and register file.

In this lab:

- You will complete the given templates for each component: instruction unit, decode unit, register file and execution unit
- You will simulate the behavior of each of the individual units using ModelSim software by using the given testbenches.
- Additionally, you will learn about the instruction formats and ALU operations.

In the next lab:

- You will complete the processor design by adding an instruction memory and data memory, integrating all the components, and performing comprehensive testing of the processor.

Lab Equipment and Software

Computer with ModelSim software installed.
Text editor and Verilog Integrated Development Environment (IDE)

Pre-lab Preparation

1. Before coming to the lab, review the following.
   - Pipelining concept
   - Instruction formats
   - ALU operations
   - Being familiar with the Verilog hardware description language.

2. Read the system specification below and answer the pre-lab design related questions on Brightspace.
3. Complete Verilog code templates before the lab starts.

Overall System Specifications

- 4-stage pipelined RISC processor
- Instruction set consisting of 16 instructions (including load, store, NOP, arithmetic, logical, and shift instructions)
- Register file with 8 general-purpose 8-bits registers.
- ALU (Arithmetic Logic Unit) for executing arithmetic, logic, and shift operations.
- Assume instruction memory has only 32 locations.
- Assume data memory has only 16 locations.
- No need to design memories in this lab.

The above information are very important when you make decisions on inputs, outputs, and signal types and sizes.

Functional Description for each unit

In this task, you will design the functional behavior and operation of each unit in the pipelined RISC processor. Your task is to design the following units. Block diagram for each unit has been given in

figure 1. You must specify the size of the signals based on the given specifications and instructions field. Take attention to the input/output signals from each unit. In the next lab, you will need to connect these blocks using structural programming style. Each unit may receive some signals from other blocks and pass some signals to the next building block. As an example, the register file receives operand *a* and *b* addresses (*opnda_addr*, and *opndb_addr*) from the decoder unit and outputs proper operands that are passed to the execution unit (*operand_a* and *operand_b*).

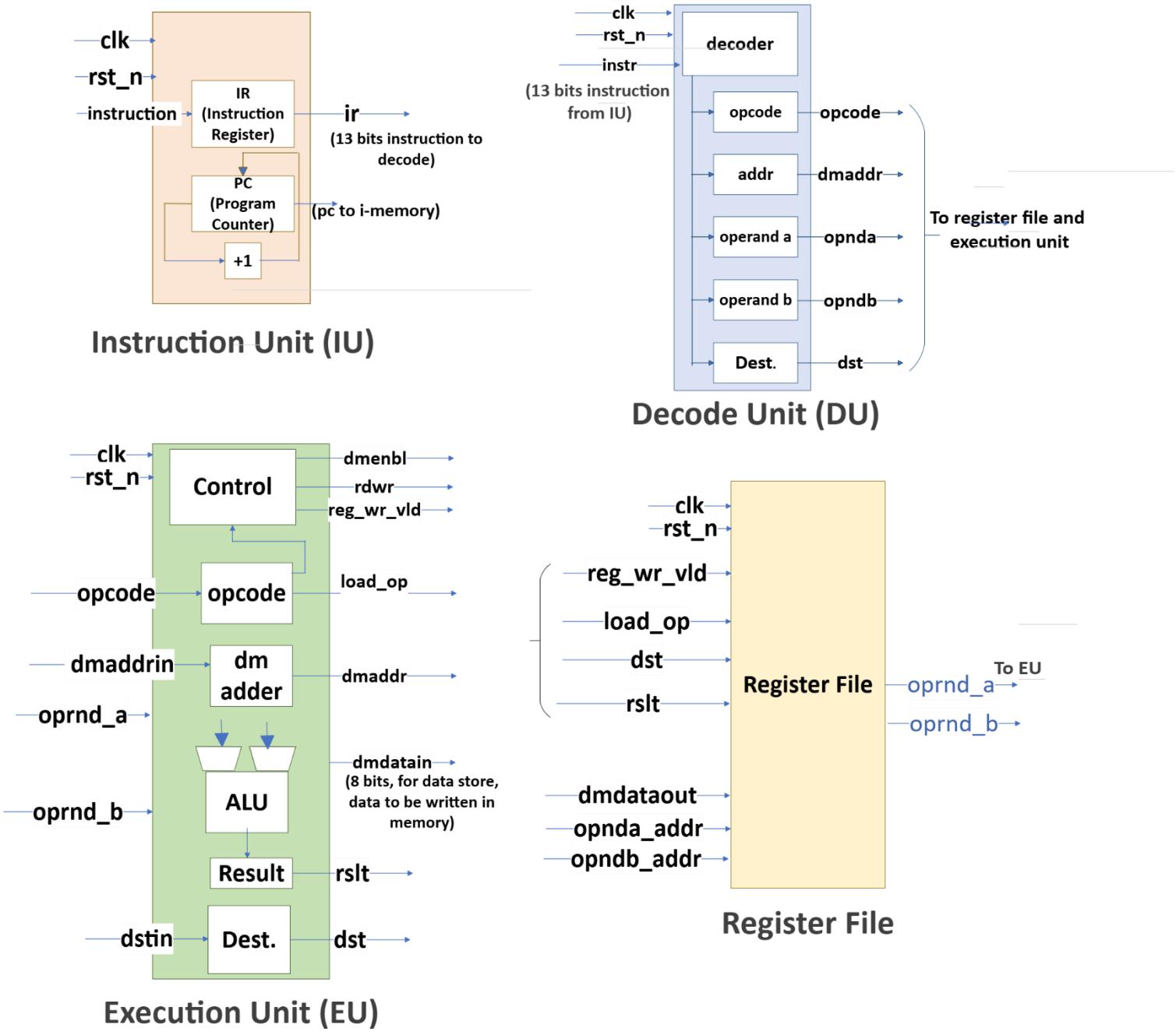

Figure 1_design units for the RISC processor

Instruction Unit: The instruction unit is responsible for fetching instructions from memory and managing the program counter (PC).

- Design an instruction register to store the fetched instruction.

- Implement a program counter that holds the address of the next instruction to be fetched.
- Make sure the instruction unit increments the program counter after each instruction is fetched.

Instruction Decode: The instruction decode unit is responsible for decoding each instruction by extracting various parts from the instruction, such as registers and immediate values.
Define the necessary logic to extract the opcode and other fields from the instruction.
Implement the logic to identify the instruction type and determine the required operands.

We are assuming the following formats given in figure.2 for the instructions. Opcodes for different instructions are given in Table.1. As an example, according to the following formats, bits 0-2 represent destination for arithmetic and logical instructions, and bits 3-5 and 6-8 represent operand B and A respectively. The opcode is stored in bits 9-12. The decode unit is responsible for extracting this information and provide it to the execution unit.

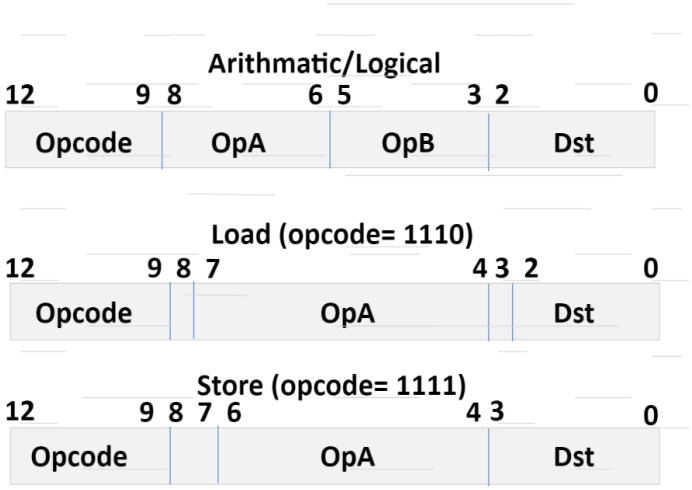

Figure 2_instructions format

Table 1-instructions opcode

instruction	opcode
ld	1110
st	1111
add	0001
sub	0010
and	0011
or	0100
xor	0101
inc	0110
dec	0111
not	1000
neg	1001
shr	1010
shl	1011
ror	1100
rol	1101

nop	0000

Instruction Execution: The instruction execution unit performs the **actual operation** specified by the instruction (opcode), involving the ALU and other necessary components.
Design an ALU to perform arithmetic, logical, and shift operations based on the instruction.
Implement the necessary logic to handle different instruction types and execute the appropriate operation.
Consider the ALU control signals and input/output requirements for different instructions. The ALU must be able to perform nop (no operation), add (addition), sub (subtraction), and, or, xor, inc (increment), dec (decrement), not (invert every bit), neg (2's complement), shr (shift to the right), shl (shift to the left), ror (rotate right), rol (rotate to the left). The definition of the instructions are given below:

Nop**:** No operation is performed
Add: Operand *A* plus operand *B*
Sub: Operand *A* minus operand *B*
And: Operand *A* AND operand *B*
Or: Operand *A* OR operand *B*
Xor: Operand *A* exclusive-OR operand *B*
Inc: Increment operand *A* by 1
Dec: Decrement operand *A* by 1
Not: Form the 1s complement of operand *A*
Neg: Form the 2s complement of operand *A*
Shr: Shift right logical operand *A*
Shl: Shift left logical operand *A*
Ror: Rotate right operand *A*
Rol: Rotate left operand *A*
Ld: Load register file from memory
St: Store register file to memory

Register File: The register file is responsible for providing the required registers for each operation and updating the destination register.

- Design a register file that stores the processor's 8 general-purpose 8-bits registers.
- Implement the logic to read the source registers based on the instruction's register operands.
- Data can be either the result of the ALU or a data that is read from the memory. Depending on the type of the instruction, either a load or arithmetic/logical, data is written to a register when the enable signal is activated.
- A register's enable signal is activated when the destination address coming from the decode block is equal to the address of the register and the write register control signal is activated.
- Include the necessary functionality to write data to the destination register.
- Make sure to define the inputs, outputs, and internal signals for each unit. Consider the control signals required for coordination between pipeline stages and proper synchronization.
- Ensure that the design of each unit aligns with the overall pipelined architecture and supports the desired instruction set.

Task 1: Complete Templates for Each Unit and its Testbench

In this task, you will implement the Verilog **module definitions (module name and interface)** for each unit designed in Task 1. Refer to the template code on Brightspace, review the code and complete the "fillout" sections. Templates for modules and testbenches are given in two files one for all modules and one for all testbenches. Examine the templates and create a separate Verilog file for each unit module and its testbench (e.g. one Verilog file for instruction unit and one Verilog file for its testbench). In the templates, look for the keyword "fillout" to find the parts that must be completed. Before you start, create

one folder and save all your files in that folder. At the end you must have 8 Verilog files (one Verilog file for each unit module and one for each unit testbench).

1.1 Instruction unit module: Create two Verilog files; one for the module and another for its testbench. Use the provided template as a reference and copy the corresponding code into these files. The risc_iunit template includes three sections to complete the module.
In the testbench, you will input an instruction (which later will come from the instruction memory), and then test if the program counter (PC) is incremented correctly. Additionally, ensure that the instruction is properly delivered to the output. You must document this in your screenshots in the next task. In the testbench you need to fillout the signal definitions and instantiations.

1.2 Instruction decode module: Create one Verilog file for the module and one Verilog file for its testbench. Refer to the template code for decode module. There are 5 sections to complete; refer to the template for explanation.
In the testbench, we will input instructions, and the decode unit must then parse these instructions, extracting the opcode, operand A, operand B, and destination addresses.

1.3 Instruction execution: Similar to the previous modules, create one Verilog file for the module and one Verilog file for its testbench. Refer to the template code for the code. In the testbench, we will perform all operations and display the outputs.

1.4 Register file unit: Create one Verilog file for the module and one Verilog file for its testbench. Refer to the template code for decode module.

- In the register file module, reset the register files with the following values instead of 0.

```
regfile0 = 8'h00;
regfile1 = 8'h22;
regfile2 = 8'h44;
regfile3 = 8'h66;
regfile4 = 8'h88;
regfile5 = 8'haa;
regfile6 = 8'hcc;
regfile7 = 8'hff;
```

Task 2: Simulations

In this final task, you will evaluate the performance of each design unit by using the completed testbench from the previous task. Employ ModelSim software (or Vivado simulator) to compile and run simulations of the Verilog design and testbench. Analyze the simulation results to verify the correct operation of each unit and ensure all your design files are saved for the upcoming lab experiment.
In the next lab, you will incorporate the instruction memory and data memory and integrate all the design units to create a complete RISC processor.
For instance, when using the testbench for the instruction unit available on Brightspace, you should observe a waveform similar to Figure 3 for the instruction unit. In this representation, values are in binary, which can be challenging to examine. To improve the waveform's readability, consider

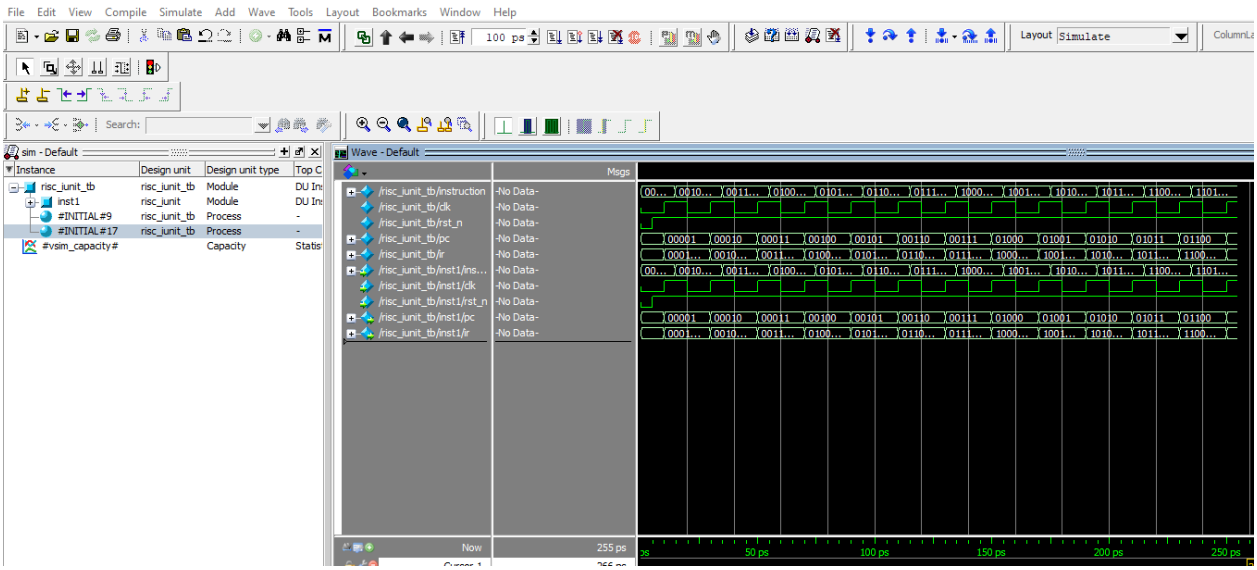

Figure 3: waveform for instruction unit test

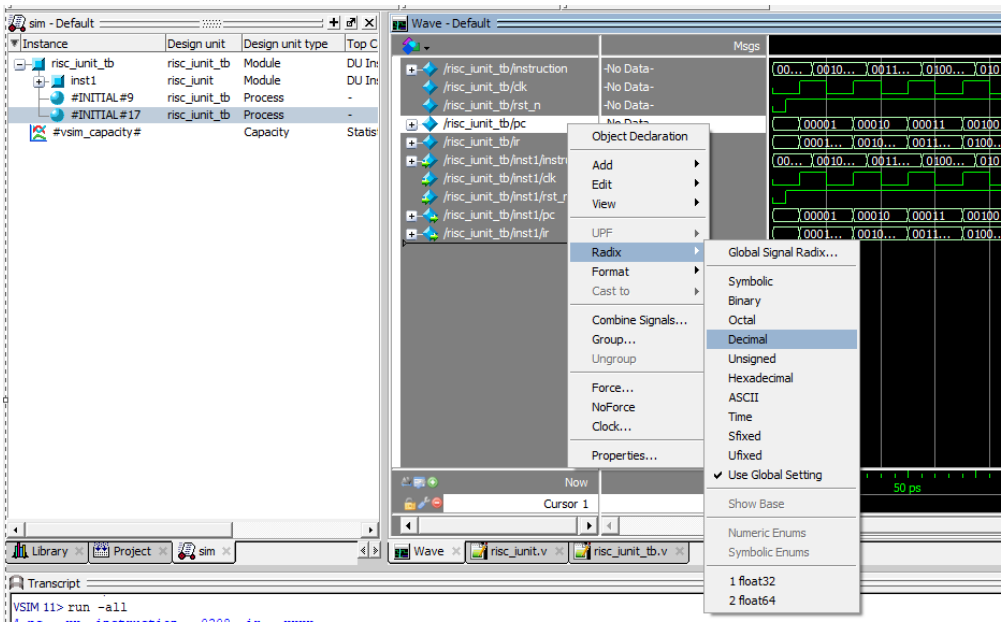

Figure 4: Changing the radix to decimal for PC register

changing the number system for the program counter and for instructions to hexadecimal. To make this change, you can right click on the signal, select radix and change it to decimal as seen in figure 4. Repeat this for instruction and change its radix to hexadecimal. The final waveform is shown in figure 5. You must make these changes to all your waveforms (all other design tests) before including them in your lab report (make this change to the PC or any values that has more than 4 bits to make it more readable). Also note that by using display command in the testbench you can display information in the transcript section of your simulation as shown in figure.6. **Make sure to include a screenshot of the display command in your report for each block.**

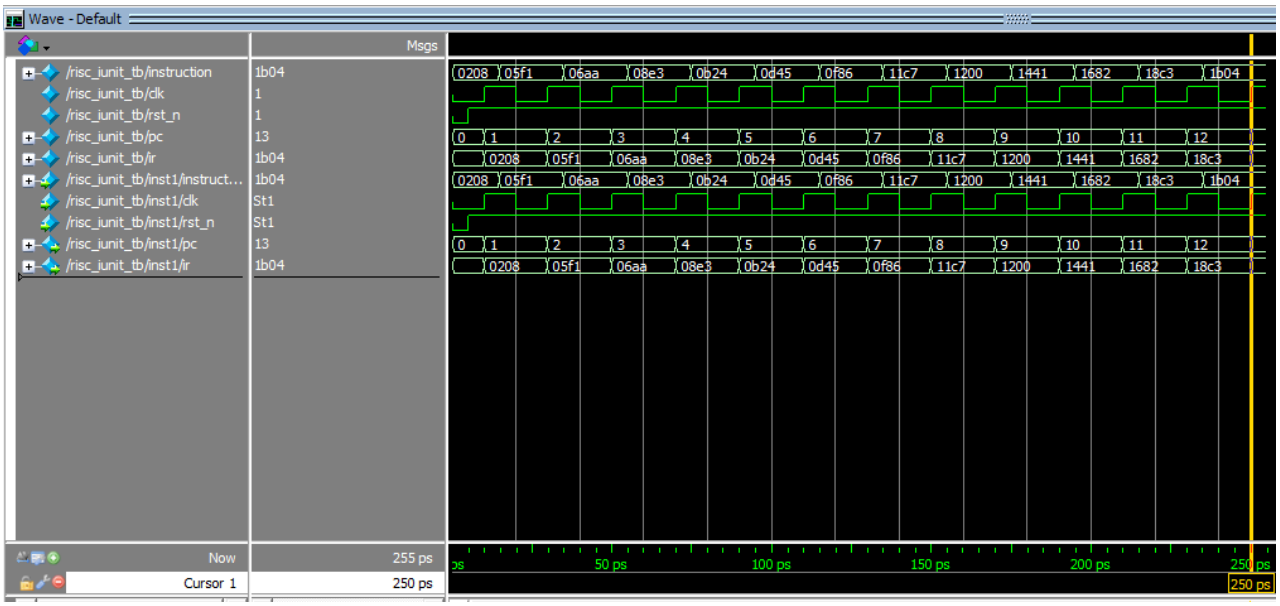

Figure 5: Final waveform for instruction design unit test

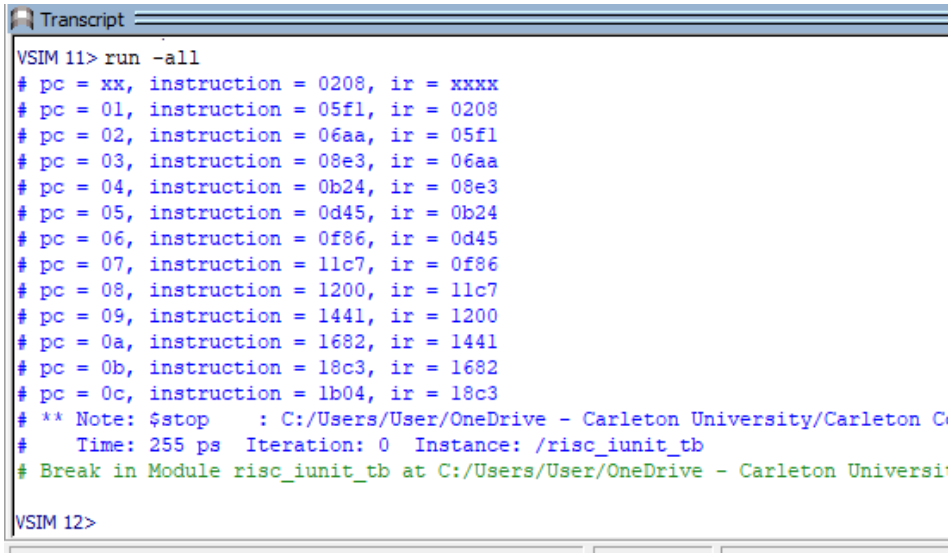

Figure 6: Transcript section

Take screenshots similar to above for each design unit operation (waveform with modified radix and transcript) and explain each unit operation in your report. Explain how the unit operates, is it what you expect based on description? Is there any variation? If yes explain why.

Note: Use hexadecimal radix for instructions, or opcodes, operands, and PC.

Reference:

Cavanagh, Joseph. *VeriLog HDL: digital design and modeling*. CRC press, 2017.

Lab Report

Write a lab report documenting your work on the design units implementation. Your report should include the following sections:

1. **Introduction**: Provide a brief introduction to the lab objective and the concepts covered.
2. **Design (Task 1-2)**: Create one section for each design unit and include the corresponding Verilog code for the module definition and implementation.
3. **Test (Task 3)**: Test the design for each operation/scenario, take one screenshot of each design unit, mark the screenshot for better understanding, analyze and explain the performance (at least two screenshots for each unit: one for the waveform and one for the transcript).
4. **Conclusion**: Summarize your experience and key takeaways (e.g. any challenges, solutions) from the lab. Reflect on what you have learned in this experiment.